

Programming Principles

Week 11: Exploring Modules, Command-Line Tools and Python's Ecosystem

What this chapter covers. This week extends Python beyond standalone interactive programs. You will learn command-line execution, shebang lines, command-line arguments and `sys.argv`, running other programs with `os.system()`, the role of compiled languages such as C, and Django as a Python web framework.

Learning outcomes

By the end of this chapter, you should be able to:

- Explain why Python's ecosystem of modules, libraries and frameworks is important.
- Run a Python script from a terminal and explain the difference between invoking Python explicitly and executing a script directly.
- Explain the purpose of a Unix/Linux shebang (`#!/`) line and the meaning of `./`.
- Explain command-line arguments and use `sys.argv` to access them.
- Explain the basic purpose of `os.system()` and the idea of reusing an existing program.
- Explain at a high level why Python can work with lower-level compiled code such as C.
- Explain what Django is and why frameworks are useful.

Reuse and integration

A programming language becomes especially useful when many people use it and build reusable software around it. Python has a large ecosystem of modules, libraries and frameworks. This means programmers can reuse existing software instead of implementing every capability themselves.

Rule Zero of programming If there is already a program that does the job, you may not need to write a new program. Reuse can save development time and reduce duplicated code.

Running Python from the command line

A common way to execute a Python script is to invoke the interpreter explicitly:

```
python3 program1.py
```

If `program1.py` contains:

```
print("Hi!")
```

the terminal displays:

```
Hi!
```

Here, the shell starts Python, and Python executes the named source file. The .py extension identifies the Python source file, while python3 identifies the interpreter.

The shebang line

Unix/Linux supports an interpreter directive called a shebang line. It begins with #! and specifies the interpreter used to execute the script.

```
#!/usr/local/bin/python3
print("Hi!")
```

With suitable executable permissions, the script can then be run directly:

```
./program2
```

The first line tells the system which interpreter should execute the file. Therefore the user does not have to type python3 before the filename.

The interpreter path is system-dependent. /usr/local/bin/python3 is an example from the lecture, not a universal Python installation path.

Why ./program?

./ means the current directory. On Unix-like systems, the current directory is not necessarily searched when a command is typed, so ./ explicitly tells the shell to execute the file from the current directory.

Command	Meaning
python3 program.py	Run program.py using Python.
./program	Execute program from the current directory.
program	Ask the shell to find program in its command search path.

Command-line arguments and sys.argv

Command-line arguments are extra words supplied when a program is started. For example:

```
./program3 Jun
```

Python provides the sys module to access these arguments.

```
import sys
print(sys.argv)
```

Conceptually, running:

```
python3 program3.py Jun
```

gives a list like:

```
["program3.py", "Jun"]
```

The key indexing rule is:

- `sys.argv[0]` is normally the program name or path used to start the program.
- `sys.argv[1]` is the first extra argument.
- `sys.argv[2]` is the second extra argument, and so on.

Worked example

```
import sys  
  
print("Hi {}!".format(sys.argv[1]))
```

Running:

```
./program3 Jun
```

produces:

```
Hi Jun!
```

Common mistake The first user-supplied argument is normally `sys.argv[1]`, not `sys.argv[0]`. Index 0 is used for the program name.

input() versus sys.argv

Approach	When information is supplied	Example
<code>input()</code>	While the program is running	<code>name = input("Name: ")</code>
<code>sys.argv</code>	When the program starts	<code>python3 hello.py Jun</code>

Use `input()` when interactive prompting is appropriate. Use command-line arguments when the program should receive its parameters at startup and work naturally with other command-line tools.

Running another program with `os.system()`

Python can ask the operating system to execute another command using `os.system()`.

```
import os  
os.system("./program3 John")
```

If `program3` prints a greeting, the output can be:

```
Hi John!
```

The important concept is program reuse: one program delegates part of its work to an existing program.

1. Import the `os` module.
2. Pass a command string to `os.system()`.
3. The operating system executes that command.

4. The called program performs its work.
5. Control returns to the Python program.

Practical caution `os.system()` is a simple mechanism for this course, but commands are operating-system dependent. A Unix/Linux command may not work unchanged on Windows.

Python as a scripting language

Python can be used for scripting tasks traditionally handled by shell languages such as `bash`. It can process files and text, accept command-line arguments, call other programs and automate repetitive tasks.

This connects naturally to your earlier work: a program can evolve from an interactive script into a reusable command-line tool.

Python and lower-level languages such as C

Python is widely used in data science and artificial intelligence. Neural-network training and other numerical workloads can require substantial computation. The lecture explains that computationally intensive parts can be implemented in a lower-level compiled language such as C, while Python provides the convenient high-level interface.

Python layer	Compiled/lower-level layer
Application logic and control	Performance-critical computation
Input/output and text processing	Fast numerical routines
Easy-to-use interface	Efficient implementation of expensive operations

Python therefore does not have to perform every low-level operation itself. Python modules can provide interfaces to compiled code. This is one reason the Python ecosystem can combine ease of use with high performance.

What you need to know. This topic is conceptual. You do not need to program neural networks in C. Understand why high-level Python code can use specialised compiled components.

Python modules, libraries and frameworks

This week shows several levels of reuse:

- `sys`: a standard-library module that provides access to command-line arguments and other interpreter information.
- `os`: a standard-library module providing operating-system-related facilities, including `os.system()`.
- Django: a much larger Python framework for developing web applications.

A module is reusable code that can be imported. A framework provides a larger application structure and reusable components for building a particular class of software.

Django and web development

Django is a Python web framework for developing websites and web applications, particularly applications whose content is managed using a database. It provides structure and reusable components so developers do not need to build the entire web infrastructure from scratch.

The key Week 11 idea is recognising the breadth of Python's ecosystem: small standard-library modules, external programs, lower-level compiled components and complete application frameworks can all work with Python.

Putting everything together

- Write a Python program.
- Run it from the terminal using `python3`.
- Add a shebang so it can be executed directly on Unix/Linux.
- Use `sys.argv` when startup arguments are more suitable than interactive input.
- Use an existing command-line program when it already performs a required task.
- Use Python interfaces to compiled code when performance-critical operations need lower-level implementations.
- Use a framework such as Django when building a larger web application.

The common theme is integration and reuse. Good programmers choose existing tools and appropriate abstractions instead of rebuilding everything from scratch.

Example: convert an interactive program to a command-line program

Interactive version:

```
name = input("Enter your name: ")
print("Hello " + name)
```

Command-line version:

```
import sys

name = sys.argv[1]
print("Hello " + name)
```

Run it with:

```
python3 hello.py Alex
```

Step-by-step reasoning:

1. Import `sys` because the program needs command-line arguments.
2. Read `sys.argv[1]` because index 0 is the program name.
3. Store the argument in a meaningful variable.
4. Use that value in the program.

5. Supply the value after the program name when starting the program.

Common mistakes and debugging

Mistake	Fix
Using <code>sys.argv[0]</code> as the first user argument	Remember that index 0 normally contains the program name.
Forgetting <code>import sys</code>	Import <code>sys</code> before using <code>sys.argv</code> .
Using <code>sys.argv[1]</code> without supplying an argument	Start the program with the expected argument or validate arguments if required.
Assuming the shebang path is universal	Use the interpreter path appropriate to the system.
Forgetting executable permission on Unix/Linux	Ensure the script has appropriate execute permission.
Assuming Unix commands work identically on Windows	Remember that operating systems have different command environments.
Thinking Python cannot use compiled code	Python can interface with lower-level compiled components through extension mechanisms.
Thinking Django is just a small library	Django is a larger web framework that provides application structure.

Practice activities

Activity 1: Predict `sys.argv`

Given:

```
import sys
print(sys.argv)
```

What should you expect from:

```
python3 test.py red blue
```

Answer:

```
["test.py", "red", "blue"]
```

Activity 2: Command-line greeter

Write a program that receives a name as `sys.argv[1]` and prints `Welcome, <name>!`. Strategy:

1. Import `sys`.
2. Read `sys.argv[1]`.
3. Build the output string.
4. Print the result.

Activity 3: Explain the shebang

Explain what the following line communicates:

```
#!/usr/local/bin/python3
```

Answer: it specifies the interpreter to use when the script is executed directly on a compatible Unix-like system.

Activity 4: Reuse an existing program

If another program already performs a conversion you need, explain why reusing it may be preferable to rewriting the conversion.

Answer: reuse avoids duplicate implementation effort and can make the overall solution simpler.

Activity 5: Python + C

Why might a data-science library expose a Python interface while implementing expensive computation in compiled code?

Answer: compiled code can provide efficient execution for performance-critical operations while Python provides an easier high-level interface.

Revision questions

Question	Answer
What is a command-line argument?	Information supplied to a program when it is started from the command line.
What does <code>sys.argv[0]</code> normally contain?	The program name or path used to invoke it.
Where is the first extra argument?	<code>sys.argv[1]</code> .
What is a shebang?	A first-line interpreter directive beginning with <code>#!</code> that specifies the interpreter for direct execution on Unix-like systems.
Why use <code>./</code> before a program?	It explicitly refers to the executable file in the current directory.
What does <code>os.system()</code> do?	It asks the operating system to execute a command.
Why can Python work with C code?	Python provides interfaces and extension mechanisms that allow Python programs to use compiled components.
What is Django?	A Python web framework for developing web applications, especially database-backed applications.

Quick reference

Concept	Example	Purpose
Run script	<code>python3 program.py</code>	Execute a Python file through the interpreter.
Shebang	<code>#!/usr/local/bin/python3</code>	Specify an interpreter for direct execution.
Current directory	<code>./program</code>	Execute a file in the current directory.
sys module	<code>import sys</code>	Access command-line/runtime information.
Arguments	<code>sys.argv</code>	Read command-line arguments.
First extra argument	<code>sys.argv[1]</code>	Access the first supplied argument.
External command	<code>os.system(command)</code>	Ask the OS to execute a command.
Web framework	Django	Develop Python web applications.

Week 11 checklist

- I can explain why Python's ecosystem is important.
- I can run a Python script from a terminal.
- I understand the purpose of a shebang line.
- I understand what `./` means.
- I can explain command-line arguments.
- I know the indexing rules for `sys.argv`.
- I can write a simple command-line program using `sys.argv`.
- I can explain the basic purpose of `os.system()`.
- I understand the principle of reusing an existing program.
- I understand why Python may use compiled code for expensive computation.
- I understand that Python can interface with lower-level code.
- I can explain what Django is and why frameworks are useful.